Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра ИИТ

Лабораторная работа № 2

По СПП

Вариант 4

Выполнила:
Студент 3 курса
Группы ПО-12
Езерская О.Г.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python

Задание 1

Прямоугольник, заданный длинами двух сторон – Предусмотреть возможность определения площади и периметра, а также логические методы,определяющие, является ли прямоугольник квадратом и существует ли такойпрямоугольник. Конструктор должен позволять создавать объекты с начальной инициализацией. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

```
Введите стороны прямоугольника:
Сторона a: 3
Сторона b (если квадрат, нажмите Enter): 4

Результат:
Прямоугольник 3.0 x 4.0
Площадь: 12.0
Периметр: 14.0
Это квадрат? False
Существует? True

Изменяем сторону a:
Новая сторона a: 10
Новая площадь: 40.0
```

```
class Rectangle:
    def __init__(self, side_a, side_b=None):
        if side_b is None:
            side_b = side_a


        self._side_a = side_a
        self._side_b = side_b


    @property
    def side_a(self):
        return self._side_a
    @side_a.setter
    def side_a(self, value):
```

```
    if value > 0:
        self._side_a = value
    else:
        print("Ошибка: сторона должна быть положительной")

@property
def side_b(self):
    return self._side_b

@side_b.setter
def side_b(self, value):
    if value > 0:
        self._side_b = value
    else:
        print("Ошибка: сторона должна быть положительной")

def area(self):
    return self._side_a * self._side_b

def perimeter(self):
    return 2 * (self._side_a + self._side_b)

def is_square(self):
    return self._side_a == self._side_b

def is_valid(self):
    return self._side_a > 0 and self._side_b > 0

def __str__(self):
    return f"Прямоугольник {self._side_a} x {self._side_b}"
```

```
    def __eq__(self, other):
        if not isinstance(other, Rectangle):
            return False
        return (self._side_a == other._side_a and self._side_b == other._side_b) or \
               (self._side_a == other._side_b and self._side_b == other._side_a)


# Ввод данных с клавиатуры
print("Введите стороны прямоугольника:")
a = float(input("Сторона a: "))
b_input = input("Сторона b (если квадрат, нажмите Enter): ")

if b_input == "":
    rect = Rectangle(a)
else:
    rect = Rectangle(a, float(b_input))

print("\nРезультат:")
print(rect)
print(f"Площадь: {rect.area()}")
print(f"Периметр: {rect.perimeter()}")
print(f"Это квадрат? {rect.is_square()}")
print(f"Существует? {rect.is_valid()}")

# Пример использования сеттера
print("\nИзменяем сторону a:")
rect.side_a = 10
print(f"Новая сторона a: {rect.side_a}")
print(f"Новая площадь: {rect.area()}")
```

Задание 2

Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной системы.

Система Вступительные экзамены. Абитуриент регистрируется на Факультет, сдает Экзамены. Преподаватель выставляет Оценку. Система подсчитывает средний балл и определяет Абитуриентов, зачисленных вучебное заведение.

```
Сколько факультетов? 2

Факультет 1:
Название: Машиностроий
Количество мест: 5

Факультет 2:
Название: Строительный
Количество мест: 8

Сколько абитуриентов? 2

Абитуриент 1:
Имя: Влад Новик
Выберите факультет:
```

```
Имя: Влад Новик
Выберите факультет:
1. Машиностроий
2. Строительный
Номер: 2

Абитуриент 2:
Имя: Влад Бричкалевич
Выберите факультет:
1. Машиностроий
2. Строительный
Номер: 1
```

```
Влад Новик:
Сколько экзаменов? 2

  Экзамен 1:
  Выберите преподавателя:
  1. Марго Александровна
  2. Вика Александровна
  Номер преподавателя (1 или 2): 1
  Оценка (2-5): 3
Марго Александровна поставила 3.0 Влад Новик

  Экзамен 2:
  Выберите преподавателя:
  1. Марго Александровна
  2. Вика Александровна
  Номер преподавателя (1 или 2): 2
```

```
  Номер преподавателя (1 или 2): 2
  Оценка (2-5): 5
Вика Александровна поставила 5.0 Влад Новик

Влад Бричкалевич:
Сколько экзаменов? 1

  Экзамен 1:
  Выберите преподавателя:
  1. Марго Александровна
  2. Вика Александровна
  Номер преподавателя (1 или 2): 2
  Оценка (2-5): 5
Вика Александровна поставила 5.0 Влад Бричкалевич
```

```
==================================================
РЕЗУЛЬТАТЫ ЗАЧИСЛЕНИЯ
==================================================

Факультет: Машиностроий (мест: 5)
✓ Влад Бричкалевич - средний балл: 5.0 - ЗАЧИСЛЕН

Факультет: Строительный (мест: 8)
✓ Влад Новик - средний балл: 4.0 - ЗАЧИСЛЕН
```

```
from abc import ABC, abstractmethod
class Person(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def get_role(self):
        Pass

class Applicant(Person):
    def __init__(self, name, faculty):
        super().__init__(name)
        self.faculty = faculty
        self.grades = []
        self.avg_score = 0
        self.enrolled = False

    def get_role(self):
        return "Абитуриент"

    def add_grade(self, grade):
        self.grades.append(grade)
        self.avg_score = sum(self.grades) / len(self.grades

class Teacher(Person):
    def __init__(self, name):
        super().__init__(name)

    def get_role(self):
        return "Преподаватель"
```

```
    def set_grade(self, applicant, grade):
        applicant.add_grade(grade)
        print(f"{self.name} поставила {grade} {applicant.name}")

class Faculty:
    def __init__(self, name, capacity):
        self.name = name
        self.capacity = capacity
        self.applicants = []

    def add_applicant(self, applicant):
        self.applicants.append(applicant)

class AdmissionSystem:
    def __init__(self):
        self.faculties = []

    def determine_enrolled(self):
        print("\n" + "="*50)
        print("РЕЗУЛЬТАТЫ ЗАЧИСЛЕНИЯ")
        print("="*50)

        for faculty in self.faculties:
            print(f"\nФакультет: {faculty.name} (мест: {faculty.capacity})")
            faculty.applicants.sort(key=lambda x: x.avg_score, reverse=True)

            for i, app in enumerate(faculty.applicants):
                if i < faculty.capacity:
                    app.enrolled = True
```

```
            print(f"✓ {app.name} - средний балл: {app.avg_score:.1f} - ЗАЧИСЛЕН")
        else:
            print(f"✗ {app.name} - средний балл: {app.avg_score:.1f} - НЕ ЗАЧИСЛЕН")
def main():
    print("="*50)
    print("СИСТЕМА ВСТУПИТЕЛЬНЫХ ЭКЗАМЕНОВ")
    print("="*50)

    # Преподаватели (как вы и хотели)
    teachers = [Teacher("Марго Александровна"), Teacher("Вика Александровна")]
    print(f"\nПреподаватели: {teachers[0].name} и {teachers[1].name}")

    # Создаем факультеты
    faculties = []
    num_faculties = int(input("\nСколько факультетов? "))

    for i in range(num_faculties):
        print(f"\nФакультет {i+1}:")
        name = input("Название: ")
        capacity = int(input("Количество мест: "))
        faculties.append(Faculty(name, capacity))

    applicants = []
    num_applicants = int(input("\nСколько абитуриентов? "))

    for i in range(num_applicants):
        print(f"\nАбитуриент {i+1}:")
        name = input("Имя: ")
```

```
    print("Выберите факультет:")
    for j, f in enumerate(faculties):
        print(f"{j+1}. {f.name}")

    faculty_choice = int(input("Номер: ")) - 1
    applicant = Applicant(name, faculties[faculty_choice])
    faculties[faculty_choice].add_applicant(applicant)
    applicants.append(applicant)

print("\n" + "="*50)
print("ЭКЗАМЕНЫ")
print("="*50)

teacher_index = 0
for applicant in applicants:
    print(f"\n{applicant.name}:")
    num_exams = int(input("Сколько экзаменов? "))

    for j in range(num_exams):
        print(f"\n  Экзамен {j+1}:")
        print("  Выберите преподавателя:")
        print(f"  1. {teachers[0].name}")
        print(f"  2. {teachers[1].name}")

        teacher_choice = int(input("  Номер преподавателя (1 или 2): ")) - 1
        grade = float(input("  Оценка (2-5): "))

        teachers[teacher_choice].set_grade(applicant, grade)
system = AdmissionSystem()
system.faculties = faculties
```

```
    system.determine_enrolled()


if __name__ == "__main__":
    main()
```

Вывод: закрепили базовые знания языка программирования Python при решении практических задач